

## Parcial refactoring sin fecha

- I. Código duplicado: En las líneas 23 y 34 se repite el código.
- II. Aplico Extract Method
  - A. Creo un método llamado “kilometrosRecorridos”.
  - B. Muevo el código de las líneas 23 y 34 al método “kilometrosRecorridos”.
  - C. Elimino la variable temporal “kilometrosRecorridos” y retorno el resultado de la operación.
  - D. Reemplazo las llamadas de la variable temporal “kilometrosRecorridos” por el método “kilometrosRecorridos” en la clase “Renta”

III.

```
protected int kilometrosRecorridos()  
{  
    return vehiculo.getKilometraje() - this.kilometrajeInicial;  
}
```

- I. Código duplicado: En las líneas 25 y 35 hay código duplicado.
- II. Aplico Extract Method
  - A. Creo un método llamado “calcularPrecioKilometro” en la clase “Renta”.
  - B. Muevo el código repetido al método “calcularPrecioKilometro”.
  - C. Reemplazo las líneas de código repetido por la llamada del método “calcularPrecioKilometro”

```

27● protected double calculaPrecioKilometro()
28 {
29     return this.kilometrosRecorridos() * vehiculo.getPrecioPorKm();
30 }
31
32● public double calcularTotal()
33 {
34     if(this.tipoRenta == "BASICO")
35     {
36         double precio = diasRenta * vehiculo.getPrecioDia() + this.calculaPrecioKilometro();
37         double adicional = 1;
38         //Los autos más viejos tienen un 15% de descuento
39         if(vehiculo.getAntiguedad() > 5)
40             adicional = 0.85;
41         return precio * adicional;
42     }
43     else if (this.tipoRenta == "PLUS")
44     {
45         return this.calculaPrecioKilometro();
46     }
47     else
48         return diasRenta * vehiculo.getPrecioDia();
49 }
50 }

```

III.

- I. Código duplicado: En las líneas 36 y 48 hay código duplicado
- II. Aplico Extract Method
  - A. Creo un método llamado "calcularPrecioDia" en la clase "Renta".
  - B. Muevo el código repetido de las líneas 36 y 48 al método "calcularPrecioDia".
  - C. Reemplazo el código repetido de las líneas 36 y 48 por la llamada al método "calcularPrecioDia".

```

22●   protected int kilometrosRecorridos()
23   {
24       return vehiculo.getKilometraje() - this.kilometrajeInicial;
25   }
26
27●   protected double calculaPrecioKilometro()
28   {
29       return this.kilometrosRecorridos() * vehiculo.getPrecioPorKm();
30   }
31
32●   protected double calcularPrecioDia()
33   {
34       return diasRenta * vehiculo.getPrecioDia();
35   }
36
37●   public double calcularTotal()
38   {
39       if(this.tipoRenta == "BÁSICO")
40       {
41           double precio = this.calcularPrecioDia() + this.calculaPrecioKilometro();
42           double adicional = 1;
43           //Los autos más viejos tirnen un 15% de descuento
44           if(vehiculo.getAntigüedad() > 5)
45               adicional = 0.85;
46           return precio * adicional;
47       }
48       else if (this.tipoRenta == "PLUS")
49       {
50           return this.calculaPrecioKilometro();
51       }
52       else
53           return this.calcularPrecioDia();
54   }
55 }

```

III.

- I. Envidia de atributos: En la línea 27 a 30 hay envidia de atributos, debe ser responsabilidad del auto.
- II. Aplico Move Method
  - A. Muevo el método “calcularPrecioKilometro” de la clase “Renta” a la clase “Vehiculo”
  - B. Modifico el método “calcularPrecioKilometro” en la clase “Vehiculo” para que reciba un parámetro de tipo entero y reemplazo la llamada del método “kilometrosRecorridos” por el parámetro recibido
  - C. En la clase “Renta” reemplazo la llamada del método “calcularPrecioKilometro” por “vehiculo.calcularPrecioKilometro” y le paso como parámetro el método “kilometrosRecorrido”
- III. En la clase Vehiculo

```
protected double calculaPrecioKilometro(int kilometros)
{
    return kilometros * this.getPrecioPorKm();
}
```

En la clase Renta

```
27● protected double calcularPrecioDia()
28 {
29     return diasRenta * vehiculo.getPrecioDia();
30 }
31
32● public double calcularTotal()
33 {
34     if(this.tipoRenta == "BASICO")
35     {
36         double precio = this.calcularPrecioDia() + vehiculo.calculaPrecioKilometro(this.kilometrosRecorridos());
37         double adicional = 1;
38         //Los autos más viejos tienen un 15% de descuento
39         if(vehiculo.getAntiguedad() > 5)
40             adicional = 0.85;
41         return precio * adicional;
42     }
43     else if (this.tipoRenta == "PLUS")
44     {
45         return vehiculo.calculaPrecioKilometro(this.kilometrosRecorridos());
46     }
47     else
48         return this.calcularPrecioDia();
49 }
50 }
```

- I. Envidia de atributos: En la línea 27 a 30 hay envidia de atributos, debe ser responsabilidad del auto.
- II. Aplico Move Method
  - A. Muevo el método “calcularPrecioDia” de la clase “Renta” a la clase “Vehiculo”.
  - B. Modifico el método “calcularPrecioDia” en la clase “Vehiculo” para que reciba un parámetro de tipo entero.
  - C. En la clase “Renta” reemplazo la llamada del método “calcularPrecioDia” por “vehiculo.calcularPrecioDia” y le paso como parámetro la variable “diasRenta”.
- III. En la clase Vehiculo

```
protected double calcularPrecioDia(int diasRenta)
{
    return diasRenta * this.getPrecioDia();
}
```

En la clase Renta

```

public double calcularTotal()
{
    if(this.tipoRenta == "BASICO")
    {
        double precio = vehiculo.calcularPrecioDia(diasRenta) + vehiculo.calculaPrecioKilometro(this.kilometrosRecorridos());
        double adicional = 1;
        //los autos más viejos tienen un 15% de descuento
        if(vehiculo.getAntiguedad() > 5)
            adicional = 0.85;
        return precio * adicional;
    }
    else if (this.tipoRenta == "PLUS")
    {
        return vehiculo.calculaPrecioKilometro(this.kilometrosRecorridos());
    }
    else
        return vehiculo.calcularPrecioDia(diasRenta);
}

```

- I. Switch Statements: En el método “calcularTotal” de la clase “Renta” hay sentencias condicionales que contienen lógica para diferentes tipos de objetos.
- II. Aplico Replace Conditional With Polymorphism.
  - A. Creo una interfaz llamada “TipoRenta” y defino un método llamado “calcularTotal”.
  - B. Creo 3 clases llamadas “Basico”, “Plus” y “KilometrajeLibre” que implementan la interfaz “TipoRenta”.
  - C. Muevo el código de cada sentencia switch de la clase “Renta” a su clase correspondiente. Por ejemplo: Si el tipo de renta es “BASICO”, muevo esa parte de código a la clase “Basico”.
  - D. Agrego 3 parámetros al método “calcularTotal” de la interfaz de “TipoRenta”, uno para el vehiculo, otro para los días de renta y otro parámetro para los kilometros recorridos.
  - E. En la implementación del método “calcularTotal” de las clases “Basico”, “Plus” y “KilometrajeLibre”, reemplazo las llamadas de vehículo, días de renta y kilómetros recorridos por los parámetros del método.
  - F. En la clase “Renta” cambio el tipo de la variable “TipoRenta” de “String” a “TipoRenta” y modifico la inicialización del constructor.
  - G. En el método “calcularTotal” de la clase “Renta”, elimino todo el código y llamo al método “calcularTotal” de la variable “tipoRenta” pasandole como parametro la variable “vehiculo”, “diasRenta” y el método “kilometrosRecorridos”

## Clase Renta

```
3 public class Renta {
4     private Vehiculo vehiculo;
5     private Cliente cliente;
6     private int diasRenta;
7     private TipoRenta tipoRenta;
8     private int kilometrajeInicial;
9
10    public Renta(Vehiculo vehiculo, Cliente cliente, int diasRenta) {
11        this.vehiculo = vehiculo;
12        this.cliente = cliente;
13        this.diasRenta = diasRenta;
14        this.kilometrajeInicial = vehiculo.getKilometraje();
15        this.tipoRenta = new Basico();
16    }
17
18    public void setTipoRenta(TipoRenta tipoRenta) {
19        this.tipoRenta = tipoRenta;
20    }
21
22    protected int kilometrosRecorridos()
23    {
24        return vehiculo.getKilometraje() - this.kilometrajeInicial;
25    }
26
27    public double calcularTotal()
28    {
29        return this.tipoRenta.calcularTotal(vehiculo, diasRenta, this.kilometrosRecorridos());
30    }
31 }
```